
Practical AI for Academics

Day 1 — an introduction to Generative AI

Claes Bäckman

Leibniz Institute for Financial Research SAFE

Let's start with an example – building a website for a paper

Three messages: agentic AI is here, the workflow matters more than the model, and verification is key.

1 The landscape has shifted

AI tools have moved from **chat** to **reasoning** to **agentic** systems that read files, run code, and iterate.

Implication: tools you tried in 2023 are not the same tools today.

2 Value is in the workflow

The agent's leverage comes from **context** (your files), **standing instructions** (CLAUDE.md), and **reusable skills** — not from raw model IQ.

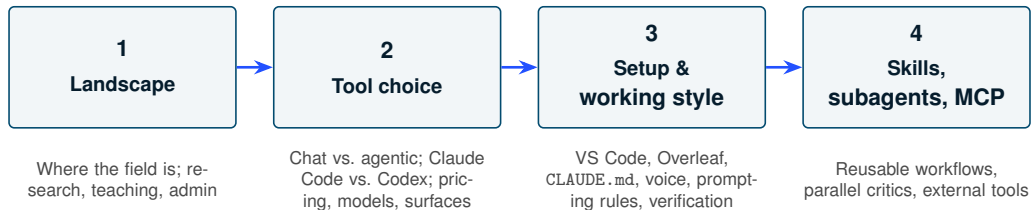
Implication: a one-hour setup compounds.

3 Verification is the new skill

Output volume is no longer the constraint — **judging it** is. Silent errors and confident statements with no backing are what you should look out for.

Implication: the researcher's edge is critical reading

Day 1 agenda



Tomorrow (Day 2). Hands-on with your own materials: a referee report on your draft, a `voice.md`, admin tasks, slides and websites.

Slides, skills, and guides for both days: claesbackman.com/aarhus

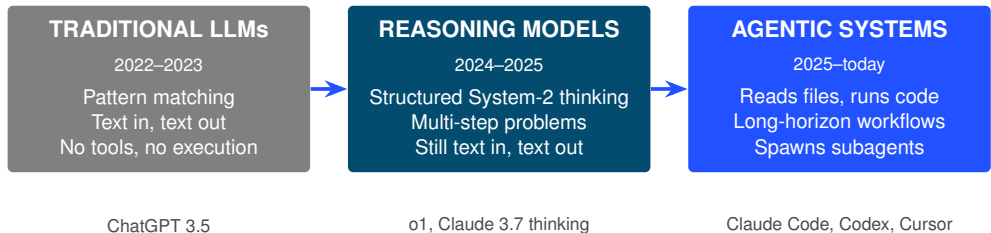
Guides: [AI guides](#), [Claude Code vs. Codex](#), [VS Code setup](#), [researcher tooling](#), [verification](#), [linking to Overleaf](#).

Who has used ChatGPT or Claude inside an IDE or terminal?
Who has used Claude Code or the Claude desktop app?

SECTION 1

The landscape

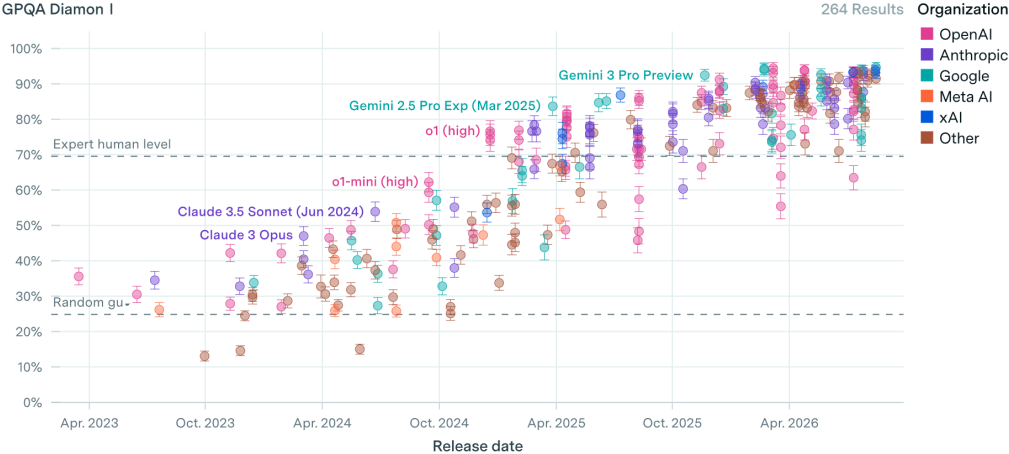
Three generations of AI models



Key takeaway. A 2023 mental model (“ChatGPT is a chatbot that hallucinates”) no longer describes what the tools can do.

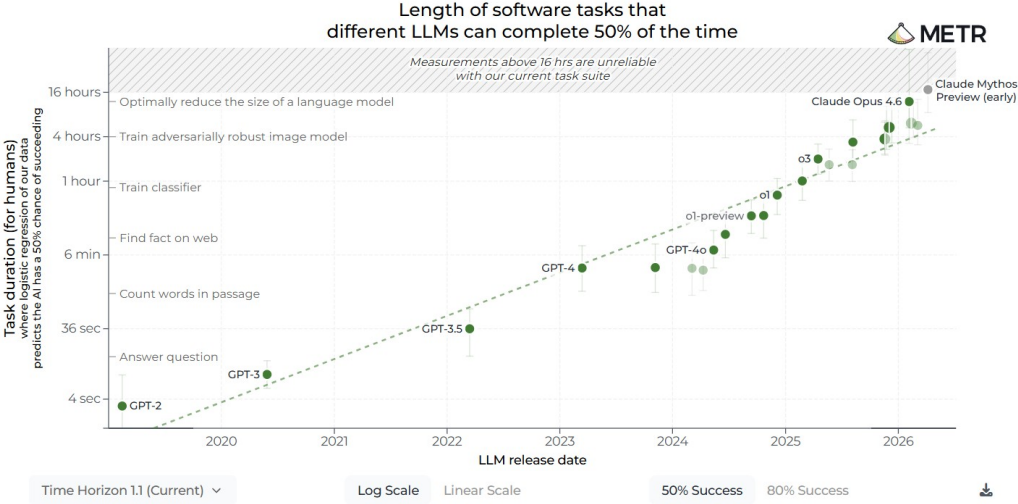
GPQA Diamond - AI performance on a set of Ph.D.-level science questions

AI performance on a set of Ph.D.-level science questions



Source: Mollick (2026); Korinek (2025); Velikov (2026).

Exponential progress

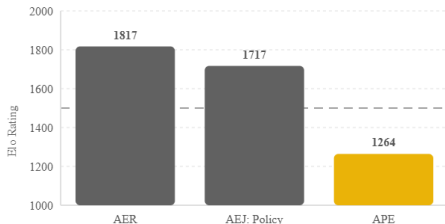


Source: Mollick (2026); Korinek (2025); Velikov (2026).

The pace is incredible

- **Novy-Marx and Velikov** (JEL paper) describes how you could automate academic research using LLMs
- **Paul Novosad** wrote a paper with AI in ~ 3 hours, and then another one with the opposite result
- **Project APE** has automated 1,000 policy-evaluation papers
- **López-Lira** has developed system that discovers a problem, generates a theory, verifies it adversarially, and writes a publication-ready paper.

Are they any good?

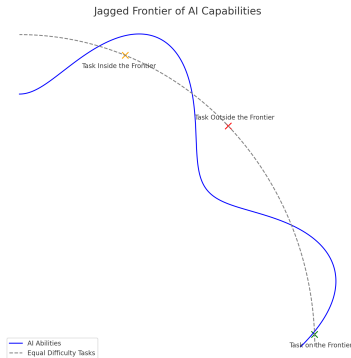


ELO ratings of papers by source.

Agents are world-class at some tasks, but not at all of them

The jagged frontier: Ethan Mollick, Dell'Acqua et al (2026)

- **Inside the frontier:** draft prose, code, lit-review summaries, structured critique, table re-derivation
- **Outside the frontier (?):** anything where the answer is not implicit in the training data, like judgment calls on how to frame a paper



The frontier is uneven — map it before you trust it.

AI is especially good at some parts of the work

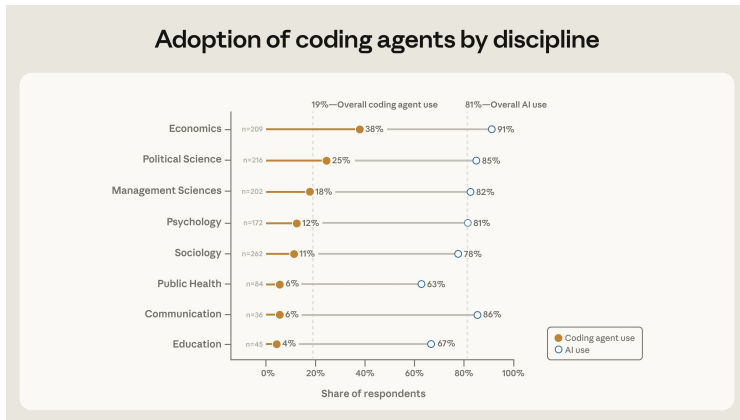
Coding is largely solved – Boris Cherny, head of Claude Code at Anthropic

Seemingly also for econometric coding

We study how large language models write code for econometric analysis. We compare three dimensions of AI-assisted coding: statistical software (Stata, R, or Python), prompting (zero-shot versus few-shot), and the degree of agency, from a chatbot that writes a single script to an agent that executes and revises its own code. On a benchmark of applied econometric and statistical tasks, moving from the chatbot to the constrained agent raises task success from 74 to 96 percent, at about eight additional cents per run. For Claude Sonnet 4.6 and GPT-5.4 through Codex, few-shot prompting improves the chatbot far more than the constrained agent, indicating that prompting and agency act as substitutes. For these models, differences across statistical software are sizeable under the chatbot but largely disappear under the constrained agent.

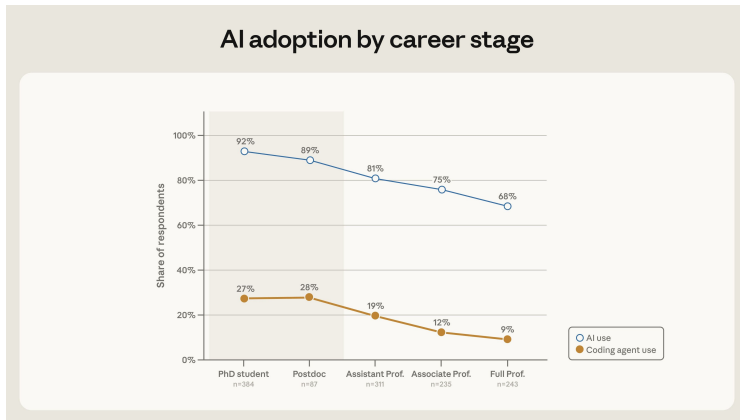
AI Agents and Prompt Engineering in Econometric Coding, [Galiani *et al.* \(2026\)](#), NBER working paper 35588,

AI adoption of chatbots is high, but not adoption of coding agents



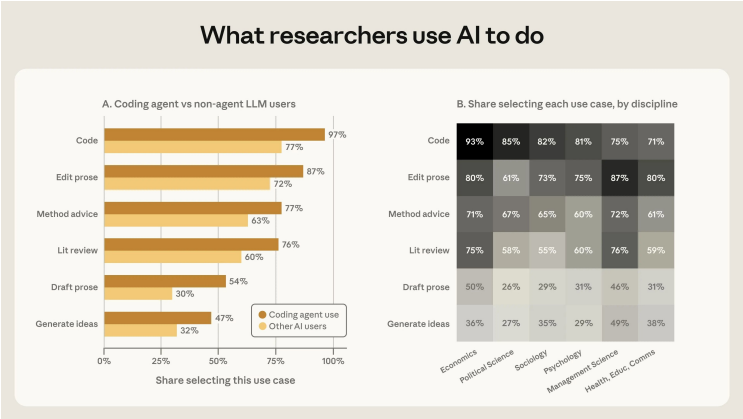
(Lyttelton *et al.* , 2026), survey of 1,260 social scientists about AI and coding agent use, fielded in February and March 2026.

Adoption also differs by career stage



(Lyttelton *et al.* , 2026), survey of 1,260 social scientists about AI and coding agent use, fielded in February and March 2026.

Common use case is coding and writing



(Lyttelton *et al.* , 2026), survey of 1,260 social scientists about AI and coding agent use, fielded in February and March 2026.

Source: Mollick (2026); Korinek (2025); Velikov (2026).

Use cases for AI

Agentic AI pays off across three distinct use cases

1 Research

- Lit-review summaries
- Pre-submission referee reports
- Feedback on your own drafts
- Responses to referees

Mental model: *AI as a feedback machine.*

2 Coding

- Stata / R / Python scripts
- Debugging error logs
- Robustness checks
- Replication packages

Mental model: *AI as a talented RA.*

3 Practical / admin / teaching

- Update slides and materials
- Create mock exams and problem sets
- Conference budgets, CV updates and websites

Mental model: *AI as a personal assistant.*

Admin is an undervalued example

Agents are great at reducing cost of administrative tasks

- **Friction is highest** — the starting cost dominates the doing
- **Quality bar is lower** — no referee, no audience
- **Nobody else to delegate to** — Small tasks not worth giving to someone else

Example — a CV for research grants

Different funders, different required formats.

Without an agent. Reformat, re-order, drop sections, rewrite the bio.

With an agent. Drop the master CV and the funder's template in the folder. The agent reformats, you check.

You can save time on a task which is easy to verify and that no one pays you for.

Resources are unequally distributed

The feedback gap

- ~40% of NBER papers acknowledge 3–5 RAs (Caro, 2024)
- Elite departments run internal seminars, lunch talks, drafts circulated to senior faculty
- Most researchers do not have access to that infrastructure
- Otherwise, feedback comes from referees — slow, sparse, end-of-process

What AI changes

- The same critic available for \$20/month, anywhere
- Pre-submission referee reports, identification critique, line-by-line edits on demand
- A more equal access to quick and relevant feedback

AI is going to change how we do research

The obvious gains

Same research, done faster and better.

- **Faster coding** — Stata/R/Python drafts, debugging error logs, robustness checks, replication packages
- **Better writing** — structured critique, voice-matched edits, pre-submission referee reports

New ways of working

New outputs and projects that were infeasible before.

- **Different ways of communicating research** — interactive websites, policy briefs, dashboards, audio summaries from one paper
- **Different types of projects** — mass automation (Project APE), automated pipelines (López-Lira), text-heavy work that used to be too costly

AI rewards expertise!

1. **Better questions.** Being an expert means that you ask better questions
2. **Steering.** Being an expert makes it easier to ask follow up questions and steer the conversation where you want it to go
3. **Validation.** Being an expert makes it easier to validate the output you get

Three things AI does not do

1. **It does not supply the question.** It lowers the cost of asking a question seriously, but it does not tell you which question is worth asking.
2. **It does not replace judgment.** You need to be in the loop to decide what to do and whether it is worth doing.
3. **It doesn't know about institutional context.** It will write a referee report on a paper if asked, but it will not know the journal policy on AI use.

Key takeaway. AI are highly capable tools that can answer any question confidently, but you are still in charge of the output.

Let's check in on the website



What an LLM actually is

What is an LLM, really?

A large language model is trained **predict the next word**, over and over, very well.

AI tools are prediction machines, not lookup machines.

- It is **not** a database — it does not look facts up, it generates plausible text
- It is **not** a calculator — it predicts what an answer *looks like*
- It is **not** connected to your data or the live web (unless the tool adds that)

A useful mental model

*A very well-read, very fast colleague who has read almost everything but remembers none of it precisely, has no access to your files, **will never admit to not knowing**, and is **unapologetic about being wrong**.*

Claude-powered AI agent's confession
after deleting a firm's entire database: 'I
violated every principle I was given'

AI agent hacks gym to get its user a spot
in pilates class

Why it sounds so confident when it's wrong

The model is trained to produce **fluent, plausible** text based on **the context it has available**.

Imagine that you are asking the LLM about a paper. If the model does not have the paper available, it may come up with something based on the training data

- It is not looking anything up (unless it has a search tool). It manufactures (predicts) what the paper is likely to say
- If the paper is well known, it may recall real facts but garble the details — coauthors, year, magnitudes
- This is a major source of hallucinations

Why it sounds so confident when it's wrong

The model is trained to produce **fluent, plausible** text based on **the context it has available**.

Imagine that you are asking the LLM about a paper. If the model does not have the paper available, it may come up with something based on the training data

- It is not looking anything up (unless it has a search tool). It manufactures (predicts) what the paper is likely to say
- If the paper is well known, it may recall real facts but garble the details — coauthors, year, magnitudes
- This is a major source of hallucinations

If you instead provide the paper in a readable form, the model reads the text and makes predictions based on **the context it has available — the paper**. It can still make mistakes, but probably fewer

Choosing your tool

Which provider to use?

The "leading" provider seem to differ month by month as new models are released

- Claude was ahead, now ChatGPT seem to be back in business
- Gemini is Google's version, Grok is from X/SpaceX/Cursor, Llama is the model from Meta

Open-weight models are cheaper to run

- Many Chinese models are cheap and not that far behind on performance
- Mistral is an European model, which may be good for regulatory compliance

In the end, switching costs are low and it is more important to just start.

Three pricing tiers for Claude

Free	\$0	Casual chat on claude.ai; no real Claude Code usage	
Pro	\$20 / mo	Light daily Claude Code use, projects, research	▶ Start here
Max	\$100–200 / mo	Heavy Claude Code use (5x / 20x); the frontier model all day	
API	Pay per token	Automation, batching, custom-built tooling	

Key takeaway. Rule of thumb: start on Pro; move to Max when usage limits start interrupting real work. Use the API only if you are building something programmatic. Codex prices the same way through ChatGPT Plus / Pro.

Four model tiers

Fast / cheap

Haiku

Conversions, summaries, bulk tasks

Default

Sonnet

Daily coding, editing, drafting

Frontier

Opus

Hard reasoning, adversarial review

Most capable

Fable

Long-running agents, hardest tasks

You can also choose effort level

Effort determines how many tokens Claude uses when responding

- Trade off between response thoroughness and token efficiency
- You can adjust the effort parameter up when you need more thorough reasoning and deeper analysis.
- Use low effort if you want to have more speed

Tokens are the unit models bill and reason in — about 3–4 characters of English. A short paper is $\approx 15\text{--}30\text{k}$ tokens. Higher effort means more tokens used.

A structural difference between chatbots and agentic tools

Chat: ChatGPT, Claude.ai

Runs in a sandbox.

- You describe the problem in words
- Paste text in, get text out
- No access to your files or your code
- Output is generic — surface-level

When to use. Brainstorming, draft polish on a single paragraph, quick lookups.

Agentic: Claude Code, Codex

Runs on your computer.

- Reads every file in your project folder
- Edits files, runs code, calls shell commands
- Calls tools: search, web, MCP servers
- Output is grounded in **your** files

When to use. Any task that touches more than one file or needs to execute code.

Quick example of the difference - a conference budget

... but the difference between chat and agentic AI getting smaller

Claude has Cowork and Claude Code in the main Claude app

ChatGPT app includes Codex, the agentic tool akin to Claude Code

And much of the workflow is converging between chat and agentic AI tools

Claude also has a desktop app for Mac and Windows

You open the app and point it at a folder — the *Code* tab is Claude Code, the *Cowork* tab is for non-coding work

The agent has the same context and does the same tasks as in VS Code

- You can see the “diffs” in the side-panel

Drawback is that you cannot see what the edits are as easily

Key takeaway. Switching costs are small once you understand the pattern. These two days use **Claude Code**, but everything transfers.

You can use whatever program you like

Surface	What it is good for
VS Code	Recommended. File tree, diff view, integrated terminal, side-by-side PDF preview.
Terminal / CLI	claude (or codex) in any shell. Good for servers and quick one-shots.
Desktop app	Claude Desktop <i>Cowork</i> / <i>Code</i> tabs, or the Codex desktop app.
Web / cloud	claude.ai/code or chatgpt.com/codex . Point at a GitHub repo.
Mobile	Capture ideas; monitor or redirect cloud agents. Not for editing.
Other editors	Cursor, JetBrains, Zed — keep your editor, add the agent.

More details at claesbackman.com/agent-ai-overview.

Claude Code vs. Codex: the workflow is the same.

Both will: read your files, run code, edit drafts, iterate, spawn subagents.

Claude Code (Anthropic)

Project memory: CLAUDE.md

Skills trigger: /review-paper

Backed by: Opus / Sonnet / Haiku

Surfaces: VS Code, CLI, web, desktop

Codex (OpenAI)

Project memory: AGENTS.md

Skills trigger: \$review-paper

Backed by: GPT-5 family

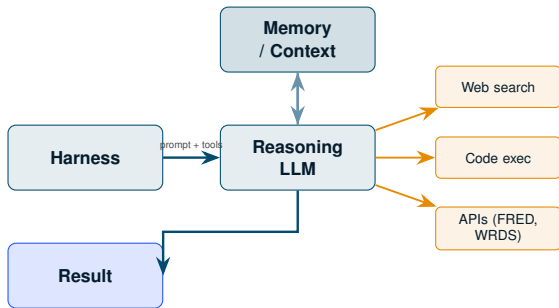
Surfaces: VS Code, CLI, web, mobile

What have you used ChatGPT or agentic AI for so far?

SECTION 5

How the tools work

The agent is the harness around the model



What happens when you type a prompt

1. Your prompt goes to the model with `CLAUDE.md`, recent files, and history
2. The model decides which **tools** to call — read, edit, shell, web
3. Each tool returns a result back into the conversation
4. The loop continues until the task is done — or the agent asks you a question

Key takeaway. ChatGPT, Claude.ai, Claude Code, Codex, and Cursor are all harnesses around **similar** models. The harness is the source of capability.

Models like Claude Code work through six core capabilities

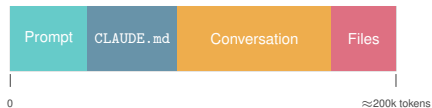
Capability	What it lets the agent do
Project files (Read)	Read files, inspect folders, understand project structure
Editing (Edit, Write)	Propose and apply diffs; create or modify files
Shell (Bash)	Run commands such as Rscript, pdflatex, tests, linters
Search (Grep, Glob)	Search filenames and file contents in the project
Web (WebSearch)	Search or fetch web pages when web access is enabled
MCP / plugins	Connect to external tools: GitHub, databases, custom servers

Context is the budget that you have available

Habits that protect it

- Point at specific files with `@filename` — don't let Claude read everything
- Start a new session for a new task (`/clear` or the + button)
- Watch the context indicator — `/context` shows the breakdown
- Convert long PDFs to markdown once, up front

Anatomy of a context window



File formats fall into three buckets

● **Native — no friction** `.md, .tex, .txt, .bib, .csv, .json, .yaml`

● **Code — native** `.py, .R, .do, .ipynb`

● **Usable — convert first** `.docx, born-digital .pdf, .xlsx`

● **Hard — OCR or skip** Scanned PDFs, `.dta / .rds` binaries, `.qsf`

Key takeaway. Convert once, commit the markdown alongside the original, point the agent at the markdown. Re-converting on every session is the single biggest waste of context.

What you can use AI for

Different use cases

Explain a paper and how it is relevant for your work

Let agents fetch data from public APIs

Write code for a paper

Update slides and materials for teaching

Help with writing and editing

Make mock exams or problem sets (students might already be doing this)

And more?

Two directions: ask it to explain and then ask it to implement.

Explain

A paper. *“Explain the identification strategy in Section 4 to a second-year PhD student. Which assumption does it rest on, and where could it fail?”*

Code. *“Walk through this do-file block by block: what does each merge do, and where could observations be dropped?”*

Instructions. *“Here are the journal’s submission guidelines. Give me a checklist of what my manuscript must satisfy and flag what is missing.”*
Same for grant calls and data-access rules.

Also: error messages, referee letters — *“what is this actually asking for?”*

Implement

From the paper. *“Implement the estimator in Section 4 in Stata, following the paper’s notation. Ask me about anything the paper leaves ambiguous before you write code.”*

From the instructions. *“Reformat the manuscript to these guidelines and list every change you made.”*

Chat vs. agent. In a chatbot you paste the code into Stata and run it yourself. An agent runs it and shows you the output.

Explain first, then implement: you check the code against an explanation you already understood.

Let the agent fetch the data — public APIs such as Statistics Denmark's StatBank.

What an API is. A URL that returns data as CSV or JSON instead of a web page.

Statistics Denmark. `api.statbank.dk` list tables, read a table's variables and codes, download the data. Packages exist (`danstat` in R, `DstApi` in Python).

Same pattern elsewhere. Eurostat, ECB, FRED, World Bank, OECD, Nationalbanken.

Verify. Agents invent table and variable codes. You need to verify, e.g. compare numbers with the website or more thorough checks.

Have the model write code to fetch the data

Prompt

“Using the Statistics Denmark API, download table FOLK1A — population by municipality — for every quarter from 2010 onwards, all 98 municipalities, totals over sex and age. Look up the table’s variables and codes first and show me what you plan to request before downloading.

Save the result as `data/raw/folk1a.csv` and write `data/README.md` describing the variables and codes. Use Python.

Then report the number of municipalities and quarters you got, and the population of Aarhus in 2020Q1 so I can check it.”

Update or create teaching materials based on your own course

Updating slides. *“These slides use 2015 examples. Suggest current replacements — recent policy episodes, updated figures to look up — keeping the structure.”*

Explaining hard concepts. *“Three ways to explain instrumental variables: intuitive, graphical, formal.”*

Slides from a source. *“Draft a 12-slide lecture outline on money demand for a second-year macro course based on Chapter 12 in @course-book, with a worked example on each concept.”*

Mock exams or problem sets

Practice problems. *“Five problems on consumer choice at increasing difficulty, with full worked solutions, based on slides in @chapter6-slides.tex.”*

Variants. *“Three numerically different versions of this problem, to rotate across years.”*

Coverage and timing. *“Map each question to a learning objective. Which syllabus topics are not tested? How long does a prepared student need?”*

Key takeaway. Verify every number. A next-token predictor is not a calculator: solve every quantitative problem yourself, or have it write code to check.

Verifying LLM output

Verification is the new bottleneck — you are looking for errors that do not crash.

Agents are fast, cheap, and competent at code, including in languages you do not know.

- They are able to produce a paper in 15 minutes and a new codebase in 20
- Or a lot more material if you allow it to run for longer. The sky is the limit

Verification is the new bottleneck — you are looking for errors that do not crash.

Agents are fast, cheap, and competent at code, including in languages you do not know.

- They are able to produce a paper in 15 minutes and a new codebase in 20
- Or a lot more material if you allow it to run for longer. The sky is the limit

But you are responsible for the output, and so you have to make sure that it is right

Verification is the new bottleneck — you are looking for errors that do not crash.

Agents are fast, cheap, and competent at code, including in languages you do not know.

- They are able to produce a paper in 15 minutes and a new codebase in 20
- Or a lot more material if you allow it to run for longer. The sky is the limit

But you are responsible for the output, and so you have to make sure that it is right

The errors that matter are not code crashes. Agents are very good at finding and fixing what crashes. What you get back is a table that runs and looks plausible — and may be wrong because the agent made the wrong decision or used the wrong fixed effect.

First principle: you have to read the code and the writing

The code is the analysis, and you are responsible once a result is in the paper.

- But you can automate some checks
- And not all code is created equal – data cleaning and regressions require more oversight than website code

Git makes this easier

Focusing on what **changed in the code** reduces the burden somewhat

`git diff` shows what changed, for you and for the agent.

- An agent that sees the whole codebase spreads its attention thin, gives generic answers, and costs tokens.
- A reviewer that sees a diff knows exactly what is new and compares it against code you already checked

Four checks that you can run

Check	What you do	What it catches	Cost
Fresh agent attacks the diff	Fresh chat or subagent reviews the diff.	Errors invisible to the agent that wrote the code	Minutes
Different model	Same diff, model from another developer (Codex vs. Claude)	Shared habits: two copies of one model make the same mistakes	Minutes
Make the agent quiz you	Fresh session explains the change, then quizzes you	Your own understanding gap	10–15 min
Reimplement in another language	Fresh folder, spec.md + README.md only, other language and model, you run it	Code that faithfully implements something other than the paper	Hours

Checks 1: a fresh agent has no stake in the code

Two ways to get independence:

- Ask the current agent: *“Use a subagent to review the code . . .”*
- Open a new chat and point it at the diff

Have the reviewer report, not fix. A reviewer that edits hands you a second diff to verify. Confident false positives are common: make it prove the findings that matter, then fix those yourself.

Check 2: Use a different model

Two instances of the same model share training data and habits of thought, so they tend to make the same mistakes.

Use a model from a different develop to break that dependency

Check 3: make the agent quiz you

The thirty-second version. Open a fresh session and ask:

“Read the last commit and the files it touches, then ask me three questions about what it changed. Do not tell me the answers until I have answered. If my answer is wrong, say so directly — do not tell me I was close.”

Always a new session, not the one that made the change. Starting fresh forces the agent to read the files instead of recalling the conversation.

A skill: /explain-diff

/explain-diff checks what changed within a Github repo and generates a html site that you can open

- **Verification.** It diffs generated tables, compares N and headline coefficients across logs, and quotes numbers only from output files it has opened.
- **Output.** One offline HTML page in five sections: what the code did before, what changed and why, consequences for the results, a code walkthrough grouped by purpose, and a five-question quiz. Saved outside the repo with a plain-text summary in the chat.

Check 04: reimplement the analysis in another language (This is an idea from Scott Cunningham)

What to do. If you write code in Stata, have the model code up results in Stata or R.

Why it is strong. If errors across languages are independent, differences in the output reveal them.

Isolate the second implementation. Enforce independence: new folder, fresh session, a `spec.md` with the specification copied from the paper (results deleted), and a `README.md` describing the raw data.

Reserve it for the main tables and for steps where a mistake is invisible in the output: a panel construction, an event-time definition, an index built from several sources.

Setting up your workspace

Five steps get VS Code + Claude Code working in under five minutes.

1. Open VS Code and the Extensions panel (Ctrl+Shift+X / Cmd+Shift+X)
2. Search “**Claude Code**” and install *Claude Code for VS Code* (extension ID `anthropic.claude-code`)
3. Restart VS Code — Claude Code opens in the sidebar
4. Sign in with your Claude account (Pro/Max/Team/Enterprise all include Claude Code) or an API key
5. Open your **full project folder** (File > Open Folder) — never a single file

Terminal alternative: `npm install -g @anthropic-ai/claude-code`, then type `claude` in any folder.

What an IDE actually is. Editor + file explorer + terminal + extensions, in one window. VS Code is the dominant free one. Think of it as Word or R Studio for code, LaTeX, markdown, and plain text.

A handful of extensions cover Stata, Python, LaTeX, and data

Stack	Extension ID	What it does
Stata	<code>tmonk.stata-workbench</code>	Run Stata from the editor; MCP server lets the agent execute commands
Python	<code>ms-python.python</code>	Syntax, IntelliSense, run button
LaTeX	<code>james-yu.latex-workshop</code>	Compile on save, side-by-side PDF preview, inline errors
Data	<code>mechatroner.rainbow-csv</code>	Colour-coded CSV columns; SQL-like filtering
	<code>tomoki1207.pdf</code>	Renders PDFs inline — read codebooks next to your code

Key takeaway. Install Python even if you do not write it. The agent uses Python as glue for many different tasks.

You can link Overleaf to your own computer and work in that folder

Why bother? Overleaf has no Claude Code, no Codex, no Copilot, no proper diff view, no terminal. The fix: write in VS Code and let Overleaf follow what you produce.

Requirement. Both routes need **Overleaf Premium** — many universities provide it through the institutional login.

Dropbox or Github?

	GitHub	Dropbox
Sync	Manual push and pull	Automatic, within seconds
History	Full git history, branches	Dropbox revisions only
Best for	Solo work, versioned drafts	Live coauthoring in Overleaf
Conflicts	Git merge	“Conflicted copy” files

Key takeaway. **Pick one per project.** Both syncs at once means duplicate-file chaos.

Two routes: GitHub Desktop for a paper trail (click to commit, no terminal), Dropbox for live coauthoring.

Setup A GitHub via GitHub Desktop

1. Overleaf: Menu → Integrations → GitHub → **Export Project to GitHub**
2. GitHub Desktop: File → Clone repository, pick the repo and a local folder
3. Open that folder in VS Code
4. Edit in VS Code. In GitHub Desktop: one-line summary, **Commit to main**, **Push origin**
5. Overleaf: Menu → GitHub → **Pull GitHub changes into Overleaf**
6. Coauthor edited in Overleaf? **Push Overleaf changes to GitHub**, then pull in GitHub Desktop

Accept the TeX .gitignore template: no .pdf or .aux in the history.

Setup B Dropbox

1. Install Dropbox and let the initial sync finish
2. Overleaf: Menu → Dropbox Sync → [Link](#). Overleaf creates Apps/Overleaf/ with one subfolder per project
3. Open the project folder in VS Code and edit as normal — every save reaches Overleaf within seconds, and Overleaf edits flow back
4. Compile in Overleaf as usual, or locally with `latexmk -pdf`

No commit step. Good for active coauthoring, less good if you want a paper trail.

Working with the agent

Every Claude Code session follows the same pattern — open the folder, set context, then prompt.

1. Open VS Code and the project folder (File → Open Folder)
2. Open the Claude Code panel in the sidebar — or the terminal (Ctrl+`) and type `claude`
3. Confirm Claude is pointed at the **full project folder** — not a single file
4. Send a first orienting prompt

First prompt to try:

“Read the files in this folder and tell me what this project is about.”

Expected behaviour. Claude reads the folder, summarises the structure, and asks you what you want to do next. **If it does not ask a question, your CLAUDE.md should tell it to.**

CLAUDE.md is the standing-instructions file — write it once, stop repeating yourself.

About me

I am a researcher in [field]. I write papers, lecture notes, policy pieces.

How I want you to work with me

- Ask clarifying questions before generating long output.
- Keep a skeptical, critical tone in feedback. Do not flatter.
- When editing, preserve my voice. Avoid generic LLM phrasing.
- Avoid bullet lists and passive voice in formal writing.
- Cite the specific line or section you are commenting on.

Things to avoid

- Do not fabricate citations.
- Do not over-claim causality when summarising empirical work.
- Do not insert emoji or markdown decorations in formal documents.

A few files hold your durable project context — know what goes where.

File	Use it for
<code>project/CLAUDE.md</code>	Project instructions. Dataset structure, sample restrictions, paper title, coauthor conventions, which script produces which output.
<code>~/.claude/CLAUDE.md</code>	Personal preferences across every project: writing style, default tone, citation conventions. Both files load; the project file wins on conflict.
<code>voice.md</code>	Writing samples. Paragraphs, abstracts, and tone notes that Claude should match when editing prose.
<code>.claude/skills/...</code>	Reusable workflows. Named tasks you want to run again without rewriting the prompt.

Key takeaway. Keep secrets and raw confidential data **out of all of them** — these files are read into context on every session and may be sent to the model provider.

Bootstrap CLAUDE.md in two minutes — type `/init`.

The built-in command: type `/init` in the Claude Code panel. Or spell it out:

“Read this project folder and draft a CLAUDE.md file with the project structure, conventions, and instructions you should follow. Do not edit anything else yet.”

What Claude does

- Reads the folder
- Infers languages, structure, conventions
- Generates a CLAUDE.md draft

Then you edit it. Add tone preferences, things to avoid, sample restrictions Claude could not infer.

Plan mode — think before you write. The agent plans, you correct, then it executes.

For anything non-trivial, ask Claude to outline its approach **before** generating.

Example prompt:

“Before writing the policy brief, outline the structure, the three findings you plan to lead with, and the one caveat you’ll flag. Do not write the brief yet. Ask me questions if you need to.”

You review the plan, correct the framing, then approve it. Claude Code has an explicit **plan mode**:

- Toggle it with `Shift+Tab` — Claude reads and proposes, but does not edit
- Accept the plan and it executes; reject it and you have lost thirty seconds

Key takeaway. Planning catches the wrong approach when it is cheap to fix — before the agent has written 800 lines you have to read.

Permissions follow a tiered model

Plan mode

Reads and proposes; cannot write.

Manual

Asks before each edit or command.

Edit automatically

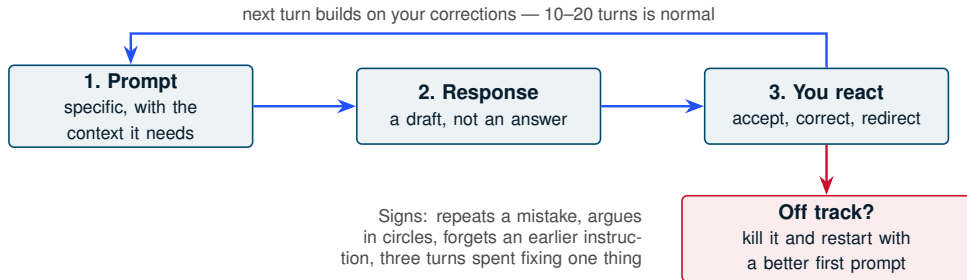
Claude will edit your selected text or the whole file

Auto mode (default mode now)

Claude will approve actions that pass a safety check and pause for anything risky

Claude Code should stay inside the folder you opened, but this is not certain. Use `/permissions` to see the current rules, and `allow-` or `block-list` specific commands (`Rscript`, `pdflatex`, `rm`) in `settings.json`.

A session is a conversation, not a query — iterate, steer, and kill it when it drifts.



Continue the conversation. The context carries your corrections, so later answers improve: *“Shorter, and use the Danish case.”* *“You misread Table 2 — look again.”* *“Now the same for Section 3.”*

Kill it when you veer off track. Sunk cost is zero. Start fresh and put what you learned into the first prompt — or into `CLAUDE.md` so you never repeat it. In Claude Code: `/clear`. In a chatbot: a new conversation.

Key takeaway. The restart is the most under-used move. A fresh session with a better first prompt beats twenty turns of repair.

Five prompting rules: specific, iterate, correct early, verify, restart.

1. **Specific.** “Feedback on my paper” is a worse request than “Give me feedback on the identification strategy in Section 4.”
2. **Iterate.** 10–20 turns, not one.
3. **Correct early.** If the first answer is off, fix it now — or restart the chat.
4. **Trust but verify.** Run the code. Re-read the passage. Check the citation.
5. **Open new sessions.** 30+ turns or the task has shifted — `/clear` and restart.

Key takeaway. The researcher who sends one long prompt and accepts the answer gets generic output. The researcher who treats the agent like a junior colleague and argues for ten turns gets work product.

Four failure modes to recognise — flattery, generic feedback, fabrication, drift.

Failure mode	Symptom	Fix
Flattery	<i>"This is a well-structured piece."</i>	Ignore. Ask for the strongest objection a hostile referee would raise.
Generic feedback	<i>"Consider strengthening the motivation."</i>	Push back: "Give me a specific sentence that would make it stronger."
Confident fabrication	Claude describes a passage that is not in the draft, or cites a paper that does not exist.	Always re-read the passage. Always check the citation.
Session drift	Repeats mistakes, invents files, gives circular answers, forgets the last three turns.	<code>/clear</code> and start fresh. Cost: 5 seconds.

End of Day 1

Tomorrow: hands-on with your own materials. Bring a draft you care about and 4–5 PDFs of your own writing.

Slides, skills, guides: claesbackman.com/aarhus

Appendix: CLAUDE.md starter template

About me

I am a researcher in [field].

How I want you to work with me

- Ask clarifying questions before generating long output.
- Critical, skeptical tone in feedback. Do not flatter.
- When editing, preserve my voice (see voice.md). No generic LLM phrasing.
- Avoid bullet lists and passive voice in formal writing.
- Cite the specific line or section you are commenting on.

Things to avoid

- Do not fabricate citations.
- Do not over-claim causality.
- Do not insert emoji or markdown decorations in formal docs.

Appendix: My skill library

Skill	Use
<code>/review-paper</code>	Pre-submission referee report, 8 agents
<code>/review-paper-light</code>	Fast 2-agent version, $\approx$ 1 minute
<code>/review-paper-code</code>	Paper + code alignment review
<code>/review-grant</code>	Pre-submission grant review, 6 agents
<code>/review-pap</code>	Pre-analysis-plan review
<code>/audit-analysis</code>	Adversarial audit of changed analysis code
<code>/explain-diff</code>	Explain a code change as an HTML page with a quiz
<code>/paper-version</code>	Paper $\rightarrow$ policy brief / 1-page / 5-page
<code>/pdf-to-markdown</code>	Convert PDFs to readable markdown

All at github.com/claesbackman/Al-research-feedback (MIT), linked from claesbackman.com/aarhus.
Fork them, modify them, make them yours.

Appendix: signs of session drift

- Claude ignores CLAUDE.md
- Repeats mistakes you already corrected
- Invents files that do not exist
- Gives circular answers
- Forgets the last three turns

Fix. Start a fresh session or close and reopen. Re-state the task. Re-attach the relevant file.

30+ turns or the task has shifted → start fresh. Always.

References I

Galiani, Sebastian, López, Federico Ariel, & Sosa, Raul A. 2026. *AI Agents and Prompt Engineering in Econometric Coding*. Tech. rept. National Bureau of Economic Research.

Lyttelton, Thomas, Massenkoff, Maxim, & Wilmers, Nathan. 2026. *Coding Agents in the Social Sciences*. Anthropic Research.